Introducción a Python

MATERIAL 2

IMPORTACIÓN Y EXPORTACIÓN DE DATOS

Material práctico

Autor:

Alexis Adonai Morales Alberto

2026

fancyhdr

Índice

1. Introducción **2**

2. Preparando ruta **2**

3. Importación de datos **3**
3.1. Función read_clipboard 3
3.2. Lectura de archivos .csv 4
3.3. Lectura de archivos .dta 7
3.4. Lectura de archivos .sav 9
3.5. Lectura de archivos .dbf 11
3.6. Lectura de archivos .xlsx 13

4. Exportación de datos **14**
4.1. Exportación en formato .csv 15
4.2. Exportación en formato .dta 16
4.3. Exportación en formato .sav 16
4.4. Exportación en formato .xlsx 17

1. Introducción

En esta sección se mostrará cómo se pueden importar y exportar datos en el espacio de trabajo de Python, mediante algunas liberías tales como: **Pandas**, **Dbf5** y **pyreadstat**.

Es importante señalar que se pueden cargar datos de diferentes tipos de archivos o extensiones, pueden ser varios, tales como: CSV; que es un tipo de datos separados por comas, o bien, también se pueden cargar datos con el formato .DTA que refiere a un espacio de almacenamiento de Stata.

Además de las dos extensiones mencionadas, en Python se pueden realizar lecturas de extensiones .SAV pertenecientes a una memoria de almacenamiento de datos de SPSS.

Y por último, tenemos un tipo de archivo que tiene la extensión de .DBF es el formato de datos utilizado originalmente por el producto dBase, siendo en la actualidad el formato más comúnmente utilizado en DBMS (Sistema de Gestión de Base de Datos), para computadoras personales.

Entonces, en resumen, se pueden importar los siguientes formatos o extensiones de bases de datos:

- Archivos con la extensión .csv
- Archivos con la extensión .dta
- Archivos con la extensión .sav
- Archivos con la extensión .dbf
- Archivos con la extensión .xlsx

De igual manera, se mostrará la forma en cómo se pueden manipular las bases y poder exportarlas en los formatos correspondientes de las extensiones anteriormente enlistadas.

Para poder lograr lo anterior, se requiere instalar la paquetería de **Pandas**, con esta librería se podrán utilizar funciones del tipo **read** con las cuales se podrán realizar las importaciones y **to** para poder exportar bases en los formatos enlistados, aunque habrá casos particulares en los cuales las instrucciones podrán cambiar un poco.

2. Preparando ruta

Es importante tener en cuenta que hay maneras particulares de importar los datos en Python, en las cuales la *ruta* también juega un papel destacado, puesto a que con esta ella se podrá cargar datos con el uso del código.

Para exportar datos, es necesario contar con un espacio o ruta definido, puesto a que se utilizarán funciones de escritura donde se podrán almacenar los archivos exportados en diferentes formatos. Cabe recalcar que solo se puede realizar mediante comando.

Es por ello que a continuación se mostrarán las formas en las cuales podrás enrutar un espacio de trabajo.

Para realizar lo anterior, solo basta con copiar la ruta dentro de nuestro sistema operativo en donde se encuentren las bases y se asigna con un nombre la dirección para que el entorno tenga entendido que de ahí se van a encontrar alamacenados las bases de datos.

A continuación se muestra el ejemplo:

[1]:
```
ruta="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/"
```

Se aprecia que como resultado se genera un objeto que contiene la ruta en donde se encuentran las bases de datos. Una vez realizado lo anterior se pueden exportar las bases de datos en las extensiones vistas.

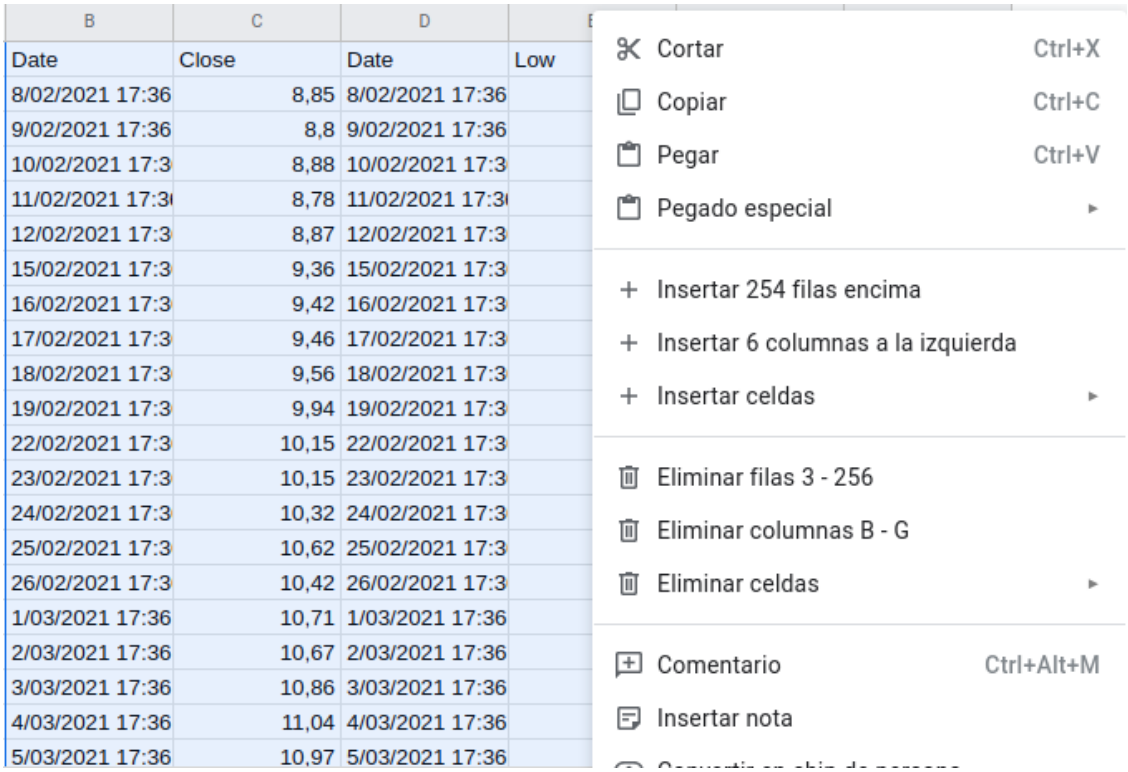
3. Importación de datos

En esta sección se mostrarán los procedimientos para importar las bases de datos en los formatos ya mencionados, entonces, para poder comenzar se hace la importación de la librería de **Pandas** la cual será parte fundamental de todos estos pasos.

[2]:
```
import pandas as pd
```

3.1. Función read_clipboard

La función de **read_clipboard()** de la librería **pandas** permite utilizar el portapapeles para importar datos a la sesión de Python, sin la necesidad de tener una ruta en especifico, lo unico que se requiere es copiar los daros mediante un sotfware que permita el copiar celdas o datos con estructuras similares a un encoding de formato UTF-8. En este caso se hará uso de una paquetería externa (puede ser excel o cualquier otra como google sheets) para poer copiar los datos, tal como se ejemplifica en la siguiente imagen:



B	C	D	E
Date	Close	Date	Low
8/02/2021 17:36	8,85	8/02/2021 17:36	
9/02/2021 17:36	8,8	9/02/2021 17:36	
10/02/2021 17:3	8,88	10/02/2021 17:3	
11/02/2021 17:3	8,78	11/02/2021 17:3	
12/02/2021 17:3	8,87	12/02/2021 17:3	
15/02/2021 17:3	9,36	15/02/2021 17:3	
16/02/2021 17:3	9,42	16/02/2021 17:3	
17/02/2021 17:3	9,46	17/02/2021 17:3	
18/02/2021 17:3	9,56	18/02/2021 17:3	
19/02/2021 17:3	9,94	19/02/2021 17:3	
22/02/2021 17:3	10,15	22/02/2021 17:3	
23/02/2021 17:3	10,15	23/02/2021 17:3	
24/02/2021 17:3	10,32	24/02/2021 17:3	
25/02/2021 17:3	10,62	25/02/2021 17:3	
26/02/2021 17:3	10,42	26/02/2021 17:3	
1/03/2021 17:36	10,71	1/03/2021 17:36	
2/03/2021 17:36	10,67	2/03/2021 17:36	
3/03/2021 17:36	10,86	3/03/2021 17:36	
4/03/2021 17:36	11,04	4/03/2021 17:36	
5/03/2021 17:36	10,97	5/03/2021 17:36	

Posteriormente, una vez copiados los datos se utilizará la siguiente sentencia:

[3]:
```
datos=pd.read_clipboard()
datos
```

[3]:
```
                    Date  Close               Date.1    Low  \
0     8/02/2021 17:36:00   8,85   8/02/2021 17:36:00   8,69
1     9/02/2021 17:36:00    8,8   9/02/2021 17:36:00   8,77
2    10/02/2021 17:36:00   8,88  10/02/2021 17:36:00   8,69
3    11/02/2021 17:36:00   8,78  11/02/2021 17:36:00   8,74
4    12/02/2021 17:36:00   8,87  12/02/2021 17:36:00   8,61
..                   ...    ...                  ...    ...
```

```
248  26/01/2022 17:36:00  11,04  26/01/2022 17:36:00  10,77
249  27/01/2022 17:36:00  11,25  27/01/2022 17:36:00  10,83
250  28/01/2022 17:36:00  11,19  28/01/2022 17:36:00  11,11
251  31/01/2022 17:36:00  11,24  31/01/2022 17:36:00  11,11
252   1/02/2022 17:36:00  11,38   1/02/2022 17:36:00  11,11

                  Date.2   High
0     8/02/2021 17:36:00   8,87
1     9/02/2021 17:36:00   8,97
2    10/02/2021 17:36:00   8,91
3    11/02/2021 17:36:00   8,91
4    12/02/2021 17:36:00   8,88
..                   ...    ...
248  26/01/2022 17:36:00  11,13
249  27/01/2022 17:36:00  11,37
250  28/01/2022 17:36:00  11,35
251  31/01/2022 17:36:00  11,29
252   1/02/2022 17:36:00  11,43

[253 rows x 6 columns]
```

[4]:
```
type(datos)
```

[4]:
```
pandas.core.frame.DataFrame
```

Se aprecia que lo realizado anteriormente da como resultado la importación de la base de datos mediante copiar y pegar con la función **read_clipboard** y el resultado es un formato *DataFrame* de la librería **pandas**.

3.2. Lectura de archivos .csv

Para cargar una base de datos con la extensión o formato CSV, se utiliza el comando **read_csv("nombre.csv")** con lo cual podremos cargar los datos de dicha base en esta extensión. Sin embargo, hay que contemplar la ruta del archivo, es decir, debemos especificar de donde se realizará la lectura, ya que solo agregar el nombre de la base no será suficiente, a menos que la base se encuentre dentro del mismo path que de la sesión de Python.

Para aclarar este punto se muestra un ejemplo:

[5]:
```
# Ruta del archivo

ruta_csv="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/
 ↪concentradohogar.csv"

# Importación del .csv

base=pd.read_csv(ruta_csv)
```

[6]:
```
base
```

[6]:
```
         folioviv  foliohog  ubica_geo  tam_loc  est_socio  est_dis    upm  \
0       100013605         1       1001        1          3        2      1
1       100013606         1       1001        1          3        2      1
2       100017801         1       1001        1          3        2      2
3       100017802         1       1001        1          3        2      2
```

```
4       100017803         1       1001        1          3        2      2
...           ...       ...        ...      ...        ...      ...    ...
89001  3260770714         1      32012        4          2      558  10149
89002  3260770715         1      32012        4          2      558  10149
89003  3260770716         1      32012        4          2      558  10149
89004  3260770717         1      32012        4          2      558  10149
89005  3260770718         1      32012        4          2      558  10149

       factor  clase_hog  sexo_jefe  ...  mater_serv  material  servicio  \
0         190          2          2  ...        0.00      0.00       0.0
1         190          2          1  ...        0.00      0.00       0.0
2         189          2          1  ...       78.68     78.68       0.0
3         189          2          1  ...        0.00      0.00       0.0
4         189          2          1  ...        0.00      0.00       0.0
...       ...        ...        ...  ...         ...       ...       ...
89001     128          2          1  ...        0.00      0.00       0.0
89002     128          2          1  ...        0.00      0.00       0.0
89003     128          2          1  ...        0.00      0.00       0.0
89004     128          2          1  ...        0.00      0.00       0.0
89005     128          2          2  ...        0.00      0.00       0.0

       deposito  prest_terc  pago_tarje   deudas  balance  otras_erog
↪smg
0          0.00        0.00         0.0     0.00     0.00         0.0  11089.
↪8
1          0.00        0.00         0.0  2360.65     0.00         0.0  11089.
↪8
2        983.60        0.00         0.0     0.00     0.00         0.0  11089.
↪8
3          0.00      885.24         0.0     0.00     0.00         0.0  11089.
↪8
4       5901.63        0.00         0.0     0.00     0.00         0.0  11089.
↪8
...         ...         ...         ...      ...      ...         ...
↪...
89001      0.00        0.00         0.0     0.00  1106.55         0.0  11089.
↪8
89002   3442.62        0.00         0.0     0.00  1425.24         0.0  11089.
↪8
89003   2213.11        0.00         0.0     0.00   879.34         0.0  11089.
↪8
89004   1377.04        0.00         0.0     0.00     0.00         0.0  11089.
↪8
89005   4426.22        0.00         0.0  2950.81     0.00         0.0  11089.
↪8

[89006 rows x 126 columns]
```

Además se guardar toda la ruta como un objeto y utilizarlo dentro del **red_csv**, también se puede utilizar, el objeto que se guardo al inicio de este documento y se la agrega + "*nombre del archivo*", tal y como se muestra a continuaciíon:

[7]:
```
# Segunda opción

base_csv=pd.read_csv(ruta+"MarginaciónMun.csv", encoding="latin-1")
base_csv
```

[7]:
```
      CVE_ENT             ENT CVE_MUN  \
0           1  Aguascalientes    1001
1           1  Aguascalientes    1001
2           1  Aguascalientes    1001
3           1  Aguascalientes    1001
4           1  Aguascalientes    1001
...       ...             ...     ...
14641       -        Nacional       -
14642       -        Nacional       -
14643       -        Nacional       -
14644       -        Nacional       -
14645       -        Nacional       -

                                                      MUN    POB_TOT      VP ␣
↪\
0                                        Aguascalientes     877190       -
1       Aguascalientes                              ...     797010       -
2                                        Aguascalientes     723043       -
3                                        Aguascalientes     643419       -
4                                        Aguascalientes     582827  121790
...                                                 ...        ...     ...
14641                                                 -  119530753       -
14642                                                 -  112336538       -
14643                                                 -  103263388       -
14644                                                 -   97483412       -
14645                                                 -   81249645       -

       ANALF  SPRIM  OVSDE  OVSEE  ... PL<5000  PO2SM  OVSD OVSDSE      IM  \
0       2.06   9.54   0.31   0.16  ...    8.73  31.13     -      -  -1.676
1       2.59  11.48   0.49   0.31  ...    8.73   29.9     -      -  -1.768
2       3.19  13.61   0.77   0.54  ...    8.21  28.37     -      -  -1.831
3       3.86  18.04      -   1.12  ...    7.67  37.24     -    1.5  -1.871
4       4.53      -      -   1.62  ...       -      -  2.12      -  -1.735
...      ...    ...    ...    ...  ...     ...    ...   ...    ...     ...
14641   5.53   16.5   2.14   0.95  ...   28.85  37.41     -      -       -
14642   6.93  19.93   3.57   1.77  ...   28.85  38.66     -      -       -
14643   8.37   23.1   5.34   2.49  ...   28.99   45.3     -      -       -
14644   9.46  28.46      -   4.79  ...   30.97  50.99     -    9.9       -
14645  12.44  29.31  21.47  12.99  ...   34.38  63.22     -      -       -

             GM IND0A100 LUG_NAC LUGAR_EST   AÑO
0      Muy bajo        -    2408        11  2015
1      Muy bajo     7.69    2409         -  2010
2      Muy bajo        -    2419        11  2005
3      Muy bajo        -    2408        11  2000
4      Muy bajo        -    2393         -  1995
...         ...      ...     ...       ...   ...
14641         -        -       -         -  2015
```

```
14642          -         -        -          -  2010
14643          -         -        -          -  2005
14644          -         -        -          -  2000
14645          -         -        -          -  1990

[14646 rows x 23 columns]
```

[8]:
```
type(base)
```

[8]:
```
pandas.core.frame.DataFrame
```

Las anteriores instrucciones generaron una importación correcta de los datos y al verificar mediante *type* se pudo comprobar que la importación se realiza en formato **dataframe**.

3.3. Lectura de archivos .dta

Para cargar una base de datos con la extensión o formato DTA, se utiliza el comando read_dta(ruta+"nombre.dta") con lo cual podremos cargar los datos de dicha base en esta extensión. Sin embargo, hay que contemplar la ruta del archivo, es decir, debemos especificar de donde se realizará la lectura, ya que solo agregar el nombre de la base no será suficiente, a menos que la base se encuentre dentro del mismo path que de la sesión de Python.

Para aclarar este punto se muestra un ejemplo:

[9]:
```
# Ruta del archivo

ruta_dta="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/
↪concentradohogar.dta"

# Importación del .dta

base_dta=pd.read_stata(ruta_dta)
```

[10]:
```
base_dta
```

[10]:
```
          folioviv foliohog ubica_geo tam_loc est_socio est_dis      upm  \
0       0100013605        1     01001       1         3     002  0000001
1       0100013606        1     01001       1         3     002  0000001
2       0100017801        1     01001       1         3     002  0000002
3       0100017802        1     01001       1         3     002  0000002
4       0100017803        1     01001       1         3     002  0000002
...            ...      ...       ...     ...       ...     ...      ...
89001   3260770714        1     32012       4         2     558  0010149
89002   3260770715        1     32012       4         2     558  0010149
89003   3260770716        1     32012       4         2     558  0010149
89004   3260770717        1     32012       4         2     558  0010149
89005   3260770718        1     32012       4         2     558  0010149

        factor clase_hog sexo_jefe  ...  mater_serv material  servicio  \
0        190.0         2         2  ...        0.00     0.00       0.0
1        190.0         2         1  ...        0.00     0.00       0.0
2        189.0         2         1  ...       78.68    78.68       0.0
3        189.0         2         1  ...        0.00     0.00       0.0
4        189.0         2         1  ...        0.00     0.00       0.0
```

```
...       ...       ...       ...  ...        ...      ...       ...
89001   128.0         2         1  ...       0.00     0.00       0.0
89002   128.0         2         1  ...       0.00     0.00       0.0
89003   128.0         2         1  ...       0.00     0.00       0.0
89004   128.0         2         1  ...       0.00     0.00       0.0
89005   128.0         2         2  ...       0.00     0.00       0.0

       deposito  prest_terc  pago_tarje   deudas  balance  otras_erog      ␣
↪smg
0          0.00        0.00         0.0     0.00     0.00         0.0  11089.
↪8
1          0.00        0.00         0.0  2360.65     0.00         0.0  11089.
↪8
2        983.60        0.00         0.0     0.00     0.00         0.0  11089.
↪8
3          0.00      885.24         0.0     0.00     0.00         0.0  11089.
↪8
4       5901.63        0.00         0.0     0.00     0.00         0.0  11089.
↪8
...         ...         ...         ...      ...      ...         ...     ␣
↪...
89001      0.00        0.00         0.0     0.00  1106.55         0.0  11089.
↪8
89002   3442.62        0.00         0.0     0.00  1425.24         0.0  11089.
↪8
89003   2213.11        0.00         0.0     0.00   879.34         0.0  11089.
↪8
89004   1377.04        0.00         0.0     0.00     0.00         0.0  11089.
↪8
89005   4426.22        0.00         0.0  2950.81     0.00         0.0  11089.
↪8

[89006 rows x 126 columns]
```

Del mismo modo que con la importación de datos en el formato **csv**, en el nombre del archivo, se puede agregar **ruta** más la extensión *.dta*, tal y como se muestra a continuación:

[11]:
```
# Opcion 2

base_dta2=pd.read_stata(ruta+"EncuestaGenero.dta",
                        convert_categoricals=False)
base_dta2
```

[11]:
```
        con1   edo   muni   loca   ageb  hr_ini1  min_ini1  hr_ter1 ␣
↪min_ter1  \
0        1.0   2.0    2.0  284.0  525-5     10.0       5.0     10.0      22.0
1        2.0   2.0    2.0  284.0  525-5     10.0       5.0     10.0      21.0
2        3.0   2.0    2.0  284.0  525-5     10.0       0.0     10.0      17.0
3        4.0   2.0    2.0  284.0  525-5     10.0      26.0     10.0      43.0
4        5.0   2.0    2.0  284.0  525-5     10.0      24.0     10.0      39.0
...      ...   ...    ...    ...    ...      ...       ...      ...       ...
1195  1196.0  30.0  108.0    1.0  137-6     11.0      40.0     12.0      18.0
1196  1197.0  30.0  108.0    1.0  137-6     12.0      33.0     13.0      12.0
```

```
1197  1198.0  30.0  108.0    1.0  137-6     13.0      27.0     14.0       5.0
1198  1199.0  30.0  108.0    1.0  137-6     14.0      20.0     14.0      59.0
1199  1200.0  30.0  108.0    1.0  137-6     15.0      14.0     15.0      52.0

      dura1  ...  sexo  edad_1  escol  cond_act  ing_ind  ing_fam  est_civil␣
 ↪ \
0      17.0  ...   2.0     4.0    4.0       2.0     -1.0      7.0        6.0
1      16.0  ...   2.0     3.0    3.0       2.0     -1.0      7.0        1.0
2      17.0  ...   2.0     3.0    4.0       2.0     -1.0      7.0        6.0
3      17.0  ...   1.0     2.0    3.0      99.0     -1.0      7.0        6.0
4      15.0  ...   2.0     3.0    3.0       2.0     -1.0      7.0        1.0
...     ...  ...   ...     ...    ...       ...      ...      ...        ...
1195   38.0  ...   2.0     3.0    3.0       2.0     -1.0      2.0        1.0
1196   39.0  ...   2.0     6.0    2.0       2.0     -1.0      3.0        1.0
1197   38.0  ...   1.0     5.0    2.0       1.0      2.0      2.0        1.0
1198   39.0  ...   2.0     4.0    2.0       2.0     -1.0      3.0        1.0
1199   38.0  ...   2.0     6.0    2.0       1.0      2.0      2.0        2.0

      total2   Pondi2  Pondi_v
0        1.0  23977.0  23960.0
1        1.0  21196.0  23960.0
2        1.0  21196.0  23960.0
3        1.0  41622.0  23960.0
4        1.0  21196.0  23960.0
...      ...      ...      ...
1195     1.0  37844.0  22780.0
1196     1.0  56712.0  22780.0
1197     1.0  97654.0  22780.0
1198     1.0  29345.0  22780.0
1199     1.0  56712.0  22780.0

[1200 rows x 311 columns]
```

[12]:
```
type(base_dta)
```

[12]:
```
pandas.core.frame.DataFrame
```

Se puede comprobar que los datos se han importado como un **data frame** pertenceciente a la librería **pandas**.

3.4. Lectura de archivos .sav

Para cargar una base de datos con la extensión o formato SAV, se utiliza el comando read_spss("nombre.sav") con lo cual podremos cargar los datos de dicha base en esta extensión. Sin embargo, hay que contemplar la ruta del archivo, es decir, debemos especificar de donde se realizará la lectura, ya que solo agregar el nombre de la base no será suficiente, a menos que la base se encuentre dentro del mismo path que de la sesión de Python.

Cabe recordar que se debe instalar la paquetería **pyreadstat** la cual permitirá la carga de esta extensión de archivos en el formaro **.sav**, para instalar solo basta con utilizar *pip install* o *conda install* según sea el caseo.

Para aclarar este punto se muestra un ejemplo:

[13]:
```
# Ruta del archivo

ruta_sav="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/
↪EncuestaGenero.sav"


base_sav=pd.read_spss(ruta_sav)
```

[14]:
```
base_sav
```

[14]:
```
        con1   edo   muni   loca   ageb  hr_ini1  min_ini1  hr_ter1 ␣
↪min_ter1  \
0        1.0   2.0    2.0  284.0  525-5     10.0       5.0     10.0      22.0
1        2.0   2.0    2.0  284.0  525-5     10.0       5.0     10.0      21.0
2        3.0   2.0    2.0  284.0  525-5     10.0       0.0     10.0      17.0
3        4.0   2.0    2.0  284.0  525-5     10.0      26.0     10.0      43.0
4        5.0   2.0    2.0  284.0  525-5     10.0      24.0     10.0      39.0
...      ...   ...    ...    ...    ...      ...       ...      ...       ...
1195  1196.0  30.0  108.0    1.0  137-6     11.0      40.0     12.0      18.0
1196  1197.0  30.0  108.0    1.0  137-6     12.0      33.0     13.0      12.0
1197  1198.0  30.0  108.0    1.0  137-6     13.0      27.0     14.0       5.0
1198  1199.0  30.0  108.0    1.0  137-6     14.0      20.0     14.0      59.0
1199  1200.0  30.0  108.0    1.0  137-6     15.0      14.0     15.0      52.0

      dura1  ...     sexo           edad_1                        escol  \
0      17.0  ...    Mujer  De 35 a 44 años  Preparatoria o Bachillerato
1      16.0  ...    Mujer  De 25 a 34 años                   Secundaria
2      17.0  ...    Mujer  De 25 a 34 años  Preparatoria o Bachillerato
3      17.0  ...   Hombre  De 15 a 24 años                   Secundaria
4      15.0  ...    Mujer  De 25 a 34 años                   Secundaria
...     ...  ...      ...              ...                          ...
1195   38.0  ...    Mujer  De 25 a 34 años                   Secundaria
1196   39.0  ...    Mujer  De 55 a 64 años                     Primaria
1197   38.0  ...   Hombre  De 45 a 54 años                     Primaria
1198   39.0  ...    Mujer  De 35 a 44 años                     Primaria
1199   38.0  ...    Mujer  De 55 a 64 años                     Primaria

        cond_act                              ing_ind  \
0     No trabaja                   No percibe ingreso
1     No trabaja                   No percibe ingreso
2     No trabaja                   No percibe ingreso
3             NC                   No percibe ingreso
4     No trabaja                   No percibe ingreso
...          ...                                  ...
1195  No trabaja                   No percibe ingreso
1196  No trabaja                   No percibe ingreso
1197  Sí trabaja  Menos de $2,047.49 (Menos de 1 SM)
1198  No trabaja                   No percibe ingreso
1199  Sí trabaja  Menos de $2,047.49 (Menos de 1 SM)

                                          ing_fam  \
0                    Más de $10,237.51 (Más de 5 SM)
1                    Más de $10,237.51 (Más de 5 SM)
```

```
2                    Más de $10,237.51 (Más de 5 SM)
3                    Más de $10,237.51 (Más de 5 SM)
4                    Más de $10,237.51 (Más de 5 SM)
...                                            ...
1195              Menos de $2,047.49 (Menos de 1 SM)
1196  De $2,047.50 a $4,095.00 (Más de 1 hasta 2 SM)
1197              Menos de $2,047.49 (Menos de 1 SM)
1198  De $2,047.50 a $4,095.00 (Más de 1 hasta 2 SM)
1199              Menos de $2,047.49 (Menos de 1 SM)

                             est_civil          total2   Pondi2  Pondi_v
0                           Soltero(a)  Total Nacional  23977.0  23960.0
1                 Unión libre/Casado(a)  Total Nacional  21196.0  23960.0
2                           Soltero(a)  Total Nacional  21196.0  23960.0
3                           Soltero(a)  Total Nacional  41622.0  23960.0
4                 Unión libre/Casado(a)  Total Nacional  21196.0  23960.0
...                                 ...            ...      ...      ...
1195              Unión libre/Casado(a)  Total Nacional  37844.0  22780.0
1196              Unión libre/Casado(a)  Total Nacional  56712.0  22780.0
1197              Unión libre/Casado(a)  Total Nacional  97654.0  22780.0
1198              Unión libre/Casado(a)  Total Nacional  29345.0  22780.0
1199  Separado(a)/Divorciado(a)/Viudo(a)  Total Nacional  56712.0  22780.0

[1200 rows x 311 columns]
```

[15]:
```
type(base_sav)
```

[15]:
```
pandas.core.frame.DataFrame
```

Se aprecia que la ejecución fue un éxito y que el formato de la importación es de un data frame elaborado con **pandas**, entonces ya se puede utilizar dicha base para hacer otra clase de ejercicios.

3.5. Lectura de archivos .dbf

Para cargar una base de datos con la extensión o formato dbf, se utiliza el comando Dbf5("archivo.dbf") de la librería **simpledbf** con lo cual se podrá cargar los datos de dicha base en esta extensión. Sin embargo, hay que contemplar la ruta del archivo, es decir, se debe especificar de donde se realizará la lectura, ya que solo agregar el nombre de la base no será suficiente, a menos que la base se encuentre dentro del mismo path que de la sesión de Python.

Posteriormente, para poder generar un data frame de la base, se utiliza la función **to_dataframe** para transformarel formato dbf en un data frame de forma correcta.

Para aclarar este punto se muestra un ejemplo:

[16]:
```
from simpledbf import Dbf5

# Se enlaza la tabla o datos en formato dbf

dbf=Dbf5(ruta+"concentradohogar.dbf")
```

[17]:
```
# Se hace la conversión a data.frame

datos_dbf=dbf.to_dataframe()
```

```
datos_dbf
```

[17]:

```
         folioviv foliohog ubica_geo tam_loc est_socio est_dis      upm  \
0      0100013605        1     01001       1         3     002  0000001
1      0100013606        1     01001       1         3     002  0000001
2      0100017801        1     01001       1         3     002  0000002
3      0100017802        1     01001       1         3     002  0000002
4      0100017803        1     01001       1         3     002  0000002
...           ...      ...       ...     ...       ...     ...      ...
89001  3260770714        1     32012       4         2     558  0010149
89002  3260770715        1     32012       4         2     558  0010149
89003  3260770716        1     32012       4         2     558  0010149
89004  3260770717        1     32012       4         2     558  0010149
89005  3260770718        1     32012       4         2     558  0010149

       factor clase_hog sexo_jefe  ...  mater_serv material  servicio  \
0         190         2         2  ...        0.00     0.00       0.0
1         190         2         1  ...        0.00     0.00       0.0
2         189         2         1  ...       78.68    78.68       0.0
3         189         2         1  ...        0.00     0.00       0.0
4         189         2         1  ...        0.00     0.00       0.0
...       ...       ...       ...  ...         ...      ...       ...
89001     128         2         1  ...        0.00     0.00       0.0
89002     128         2         1  ...        0.00     0.00       0.0
89003     128         2         1  ...        0.00     0.00       0.0
89004     128         2         1  ...        0.00     0.00       0.0
89005     128         2         2  ...        0.00     0.00       0.0

       deposito  prest_terc  pago_tarje   deudas  balance  otras_erog
↪smg
0          0.00        0.00         0.0     0.00     0.00         0.0  11089.
↪8
1          0.00        0.00         0.0  2360.65     0.00         0.0  11089.
↪8
2        983.60        0.00         0.0     0.00     0.00         0.0  11089.
↪8
3          0.00      885.24         0.0     0.00     0.00         0.0  11089.
↪8
4       5901.63        0.00         0.0     0.00     0.00         0.0  11089.
↪8
...         ...         ...         ...      ...      ...         ...
↪...
89001      0.00        0.00         0.0     0.00  1106.55         0.0  11089.
↪8
89002   3442.62        0.00         0.0     0.00  1425.24         0.0  11089.
↪8
89003   2213.11        0.00         0.0     0.00   879.34         0.0  11089.
↪8
89004   1377.04        0.00         0.0     0.00     0.00         0.0  11089.
↪8
89005   4426.22        0.00         0.0  2950.81     0.00         0.0  11089.
↪8
```

```
[89006 rows x 126 columns]
```

[18]:
```
type(datos_dbf)
```

[18]:
```
pandas.core.frame.DataFrame
```

Se demuestra que la importación de la base en formato **dbf** tuvo éxito, por lo que el formato de salida dió como resultado un **dataframe** elaborado con **pandas**, por lo tanto, se puede utilizar los datos para futuros ejercicios.

3.6. Lectura de archivos .xlsx

Para cargar una base de datos con la extensión o formato dbf, se utiliza el comando **read_excel** de la librería **pandas** con lo cual se podrá cargar los datos de dicha base en esta extensión. Sin embargo, hay que contemplar la ruta del archivo, es decir, se debe especificar de donde se realizará la lectura, ya que solo agregar el nombre de la base no será suficiente, a menos que la base se encuentre dentro del mismo path que de la sesión de Python.

A continuación se muestra un ejemplo de cómo importar una base de datos.

[19]:
```
## Opción 1

ruta_xlsx="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/
↪Barriles.xlsx"

base_xlsx1=pd.read_excel(ruta_xlsx)

base_xlsx1
```

[19]:
```
       Fecha  Barriles
0    1984/01      2629
1    1984/02      2714
2    1984/03      2667
3    1984/04      2781
4    1984/05      2801
..       ...       ...
446  2021/03      1697
447  2021/04      1693
448  2021/05      1688
449  2021/06      1698
450  2021/07      1701

[451 rows x 2 columns]
```

[20]:
```
## Opción 2


base_excel2=pd.read_excel(ruta+"TipoCambio.xlsx")

base_excel2
```

[20]:
```
          Fecha    Dolar     Euro
0    2011-01-03  12.2574  16.3881
```

```
1    2011-01-04  12.2619  16.4064
2    2011-01-05  12.2064  16.1552
3    2011-01-06  12.2256  16.0204
4    2011-01-07  12.2369  15.8737
...         ...      ...      ...
2703 2021-10-05  20.5818  23.8625
2704 2021-10-06  20.6947  23.8775
2705 2021-10-07  20.6040  23.8193
2706 2021-10-08  20.7293  23.9683
2707 2021-10-11  20.8017  24.0728

[2708 rows x 3 columns]
```

[21]:
```
type(base_excel2)
```

[21]:
```
pandas.core.frame.DataFrame
```

Con lo anterior se comprueba que a importación en el formato **xlsx** fue exitosa y en formato correcto, pues al utilizar *type* da como resultado un **dataframe** elaborado con **pandas**.

4. Exportación de datos

Para esta sección, se utilizarán columnas de una base de datos de las que anteriormente se importarón y con ella se realizarán las exportaciones en diferentes formatos, tales como:

- Extensión .csv
- Extensión .dta
- Extensión .sav
- Extensión .xlsx

Cabe recordar o resaltar que para la elaboración de estas exportaciones se requiere una ruta en donde se almacenarán las bases y además se necesitarán las librerías de: **pandas**, **Dbf5** y **pyreadstat**. A continuación se elaborará la ruta donde se exportarán los archivos.

[22]:
```
ruta2="/home/adonai/Documentos/Visualizacion de datos con Python/Bases/"
```

Seguido, se seleccionarán diversas columnas de la base_csv correspondiente a una encuesta de ingresos y gastos en los hogares que elabora el INEGI.

A continuación se muestra el proceso:

[23]:
```
base_export=base[["folioviv","foliohog","upm",
                  "est_dis", "factor", "ing_cor",
                  "educa_jefe", "sexo_jefe", "edad_jefe"]]

base_export
```

[23]:
```
         folioviv  foliohog    upm  est_dis  factor   ing_cor  educa_jefe  \
0       100013605         1      1        2     190  16229.49           9
1       100013606         1      1        2     190  31425.68           8
2       100017801         1      2        2     189  33979.16          10
3       100017802         1      2        2     189  71557.37           8
4       100017803         1      2        2     189  90703.26          10
...           ...       ...    ...      ...     ...       ...         ...
89001  3260770714         1  10149      558     128  14227.85           4
```

```
89002  3260770715         1  10149      558     128  13878.67           3
89003  3260770716         1  10149      558     128  20104.84           3
89004  3260770717         1  10149      558     128  25967.20           4
89005  3260770718         1  10149      558     128  45415.16           6

       sexo_jefe  edad_jefe
0              2         48
1              1         46
2              1         26
3              1         29
4              1         63
...          ...        ...
89001          1         65
89002          1         75
89003          1         52
89004          1         49
89005          2         64

[89006 rows x 9 columns]
```

Con la base anterior, se prosigue a realizar las exportaciones correspondientes.

4.1. Exportación en formato .csv

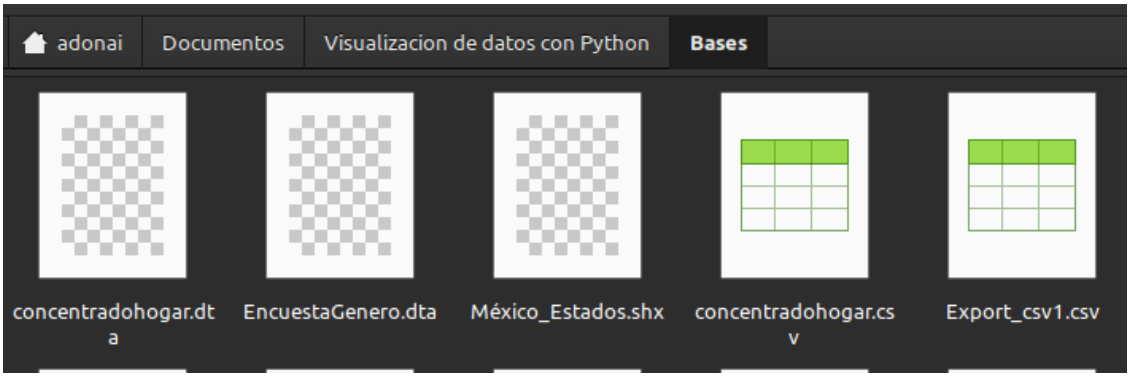
Para exportar un dataframe en formato **.csv**, se requiere usar la función **to_csv** de la librería **Pandas** lo cual permite almacenar un marco de datos en dicha extensión. Sin embargo hay que contemplar la ruta en donde se guardarán los archivos y seguido de ello especificar el nombre seguido de la extensión de la archivo con el que se desea salvar.

A continuación se muestra un ejemplo:

[24]:
```
# Opción 1: Ruta completa

base_export.to_csv("/home/adonai/Documentos/Visualizacion de datos con␣
↪Python/Bases/Export_csv1.csv",
                   index=False, encoding='utf-8')
```

Al realizar lo anterior, en la carpeta en donde se enruto o direcciono la exportación debe aparecer el archivo con el nombre de la siguiente manera:



Ahora, se mostrará otro ejemplo, anexando el objeto de ruta, seguido del nombre en el que se desea exportar el archivo.

[25]:
```
## Opción 2: Ruta + nombre

base_export.to_csv(ruta2+"Export_csv2.csv",
                   index=False,
                   encoding='utf-8')
```

4.2. Exportación en formato .dta

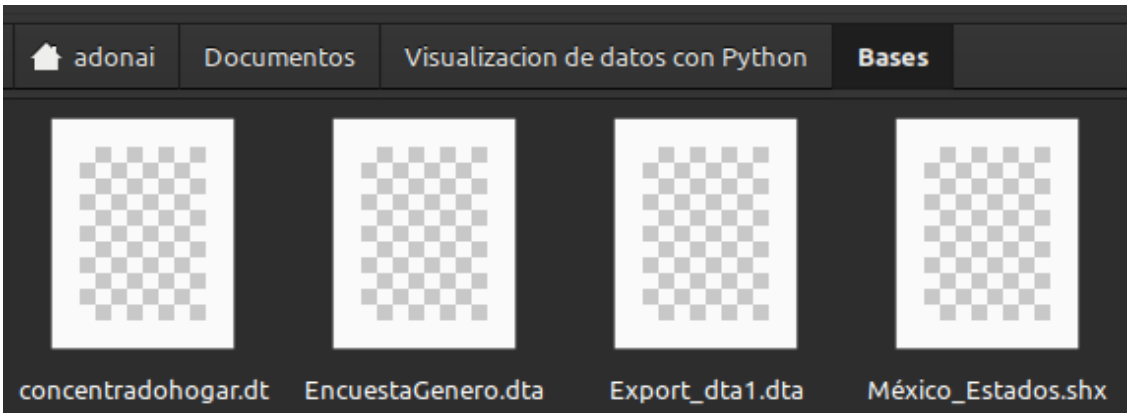
Para exportar un dataframe en formato .csv, se requiere usar la función **to_stata** de la librería **Pandas** lo cual permite almacenar un marco de datos en dicha extensión. Sin embargo hay que contemplar la ruta en donde se guardarán los archivos y seguido de ello especificar el nombre seguido de la extensión de la archivo con el que se desea salvar.

A continuación se muestra un ejemplo:

[26]:
```
## Opción 1: Ruta completa

base_export.to_stata("/home/adonai/Documentos/Visualizacion de datos con␣
 ↪Python/Bases/Export_dta1.dta",
                     write_index=False)
```

Al realizar lo anterior, en la carpeta en donde se enruto o direcciono la exportación debe debe aparecer el archivo con el nombre de la siguiente manera:



Ahora, se mostrará otro ejemplo, anexando el objeto de ruta, seguido del nombre en el que se desea exportar el archivo.

[27]:
```
## Opción 2: Ruta + nombre

base_export.to_stata(ruta2+"Export_dta2.dta",
                     write_index=False)
```

4.3. Exportación en formato .sav

Para exportar un dataframe en formato .csv, se requiere usar la función **write_sav** de la librería **pyreadstat** lo cual permite almacenar un marco de datos en dicha extensión. Sin embargo hay que contemplar la ruta en donde se guardarán los archivos y seguido de ello especificar el nombre seguido de la extensión de la archivo con el que se desea salvar.

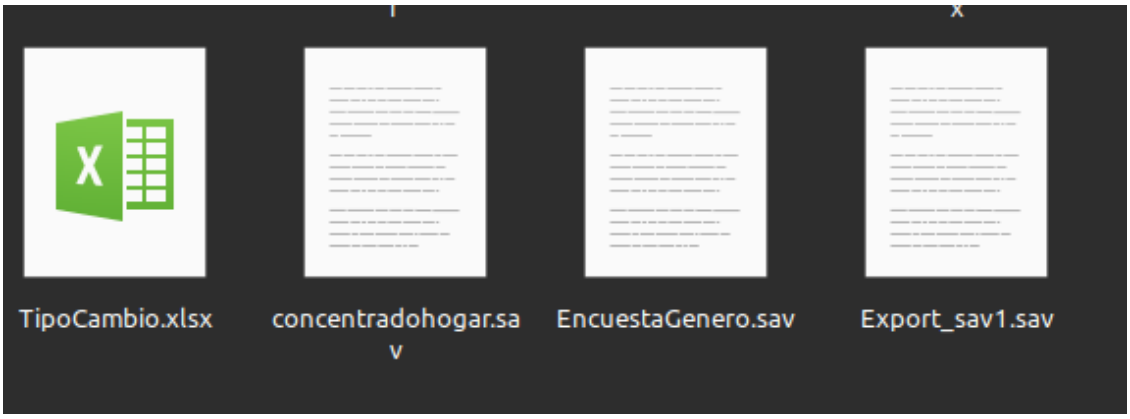
A continuación se muestra un ejemplo:

[28]:
```
## Opción 1: Ruta completa

import pyreadstat

pyreadstat.write_sav(base_export,
                     "/home/adonai/Documentos/Visualizacion de datos con␣
 ↪Python/Bases/Export_sav1.sav")
```

Al realizar lo anterior, en la carpeta en donde se enruto o direcciono la exportación debe aparecer el archivo con el nombre de la siguiente manera:



Ahora, se mostrará otro ejemplo, anexando el objeto de ruta, seguido del nombre en el que se desea exportar el archivo.

[29]:
```
## Opción 2: Ruta + nombre

pyreadstat.write_sav(base_export,
                     ruta2+"Export_sav2.sav")
```

4.4. Exportación en formato .xlsx

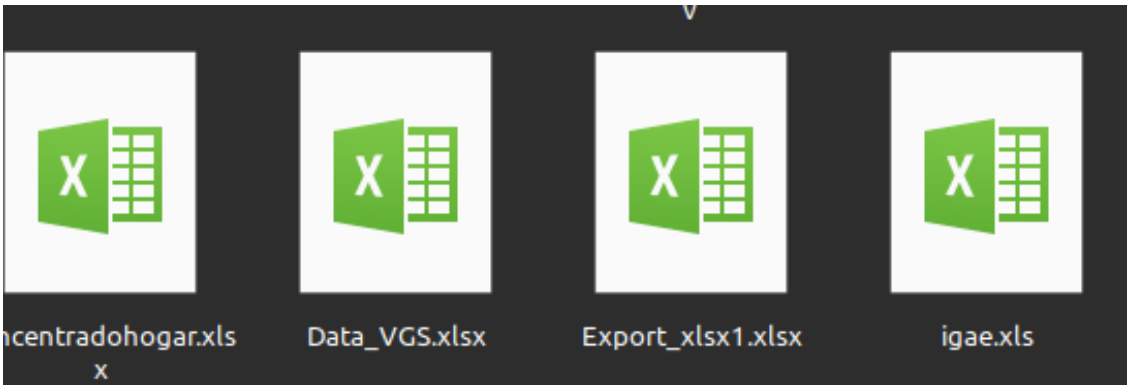
Para exportar un dataframe en formato .csv, se requiere usar la función **to_excel** de la librería **pandas** lo cual permite almacenar un marco de datos en dicha extensión. Sin embargo hay que contemplar la ruta en donde se guardarán los archivos y seguido de ello especificar el nombre seguido de la extensión de la archivo con el que se desea salvar.

A continuación se muestra un ejemplo:

[30]:
```
## Opción 1: Ruta completa


base_export.to_excel("/home/adonai/Documentos/Visualizacion de datos con␣
 ↪Python/Bases/Export_xlsx1.xlsx",
                     index=False)
```

Al realizar lo anterior, en la carpeta en donde se enruto o direcciono la exportación debe aparecer el archivo con el nombre de la siguiente manera:



Ahora, se mostrará otro ejemplo, anexando el objeto de ruta, seguido del nombre en el que se desea exportar el archivo.

[31]:
```
## Opción 2: Ruta + nombre

base_export.to_excel(ruta2+"Export_xlsx2.xlsx")
```